# Enhanced Packet Sniffer Using Python: A Lightweight Research Toolkit

Mrs. R. Swapna[0009-0007-8018-1866], Durga prasad[0009-0009-7338-9639], Ekeshwar[0009-0000-4037-6289], Naga Nandhini[0009-0008-9824-8253], Bhanu Sri Varsha[0009-0007-1890-2159], Devika[0009-0009-1932-8579]

[1] Assistant Professor, Department of Computer Science and Engineering – Artificial Intelligence, Vignan's Institute of Information Technology, Visakhapatnam, India

[2], [3], [4], [5], [6] Department of Computer Science and Engineering – Cyber Security, Vignan's Institute of Information Technology, Visakhapatnam, India

{pailaswapna2020, dpprasad2904, ekesh13899, vnnandini05, varshurao1, devikanarisipuram1706}@gmail.com

**Abstract**

In order to maintain secure and reliable digital systems, network traffic has become the fundamental requirement for monitoring. Though the sniffing tools are available, they are difficult to use and required advanced technical knowledge. This paper establishes a Python-based Enhanced Packet Sniffer that designed to ensure the network traffic analysis and also some meaning features like packet capturing, filtering of packets based on protocols etc. Here we are using Scapy for capturing of packets in network traffic and a Tkinter – a graphical interface for visualization. The Pythonbased Enhanced Packet Sniffer supports real time packet monitoring, detecting the suspicious activities in network traffic such as ARP spoofing and SYN flood attacks. In additionally, it also performs DNS query analysis by using entropy-based method and extracting the TLS certificate for inspection. The proposed solution is lightweight, flexible, especially used for educational use, experimentation, and in small-scale research purposes.

Keywords: Packet Capture, Protocol Analysis, Intrusion Detection, Anomaly Detection, Scapy, Tkinter

## 1 Introduction

Increasing of Computer Networks can lead to increasingly exposure to different types of cyber threats. Computer Network plays a crucial role n business, organizations, communication and exchange of data. Without monitoring the network, the vulnerabilities that are present in the system will be exploited by the threats that can lead to cyberattacks such as spoofing, flooding etc. Due to cyber-attacks, there will be exposure of sensitive data, reputation loss for an organization. To monitor the network traffic, we have Packet Sniffier. Packet Sniffer plays a key role to capture the data packets and monitor the network traffic. It is also used for identifying the unusual behavior, detecting the suspicious activities.

There are some well known tools like Wireshark and Snort which is used to capture the network traffic and are widely used in researches and industries mostly. But these tools are difficult to understand for the beginners. These tools are difficult for the beginners due to too many panels are present like packet list, bytes etc. which can confuse the beginners. Understanding each section takes some time. And also apply the filters (ip.addr==192.168.1.0) are powerful but it is tricky. Mistakes in the filters can lead to hide the data packet in the network traffic.

To address this gap, this work presents a Python-based Enhanced Packet Sniffer that combines simplicity with useful functionality. The system is designed to be easy to use while still supporting features such as protocol decoding, anomaly detection, DNS entropy analysis, and TLS certificate inspection. The addition of a graphical interface further improves usability by removing the need for complex command-line interaction.

The main contributions of this work are as follows:

- A packet capture model framework using Python

- Decoding of protocols such as Ethernet, IP, TCP, UDP, ICMP, ARP, DNS, and TLS in real time
- Detection of ARP spoofing and SYN flood attacks by using simple heuristic techniques
- Identification of suspicious DNS queries with the help of entropy-based analysis
- Extraction of TLS certificate information and also their examination
- A user-friendly graphical interface for monitoring and exporting data

**Related Work**

Monitoring of the Network traffic and intrusion detection have been widely happening over the years. Some of the tools are used for the detailed analysis of protocol and their capabilities, but the users who are new to the networking can face a lot of difficulties regarding the interface, configurations, real-time analysis. Similarly, Snort a tool that is used for intrusion detection that have been widely studied over the years. Snort is an effective tool but it requires the continuous configuration and the maintenance.

Python become most popular due to its flexibility and ease of development. Python provides various frameworks and Libraries. In this we use Libraries like Scapy. Scapy enables the developers to create a packet sniffers for learning and experimentation on the Network Traffic . So many of the implementations mainly focused on capturing the data packets over the network and not include the advanced features regarding the detection or user-friendly interfaces .

Some recent approaches have explored the use of machine learning techniques to identify unusual traffic patterns. While these methods show promise, they often involve high computational costs and may not be practical for lightweight or educational applications.

The system proposed in this paper aims to provide a balanced solution by combining ease of use with essential analytical capabilities. It is particularly intended for students, researchers, and small-scale experimental setups.

**Methodology**

**System Architecture**

The Enhanced Packet Sniffer is designed using a layered structure consisting of five main components:

1. **Packet Capture Layer:** This layer is responsible for collecting the network traffic by using Scapy, with raw sockets as an alternative when it is requried.
2. **Protocol Decoder Layer:** The main purpose of this layer is to Extracts and interprets data from various protocols such as Ethernet, IP, TCP, UDP, ARP, ICMP, DNS, and TLS.

3. **Analysis Module Layer: It is responsible for** Identifying the suspicious activities which includes ARP spoofing, SYN flood attacks, and unusual DNS queries in the network traffic.

4. **Visualization Layer**: In this layer user is using a Tkinter-based graphical interface. This interface is used to display the packet information, alerts, and network activity

5. **Export Functions Layer:** If the users want to save the captured data from the network, they can save in the formats like PCAP or JSON for later analysis.

## Workflow

The system follows a continuous monitoring process:

6. From the selected interface , Network traffic is captured.

7. Each packet that is captured will be decoded and categorized based on its protocol.

8. The analysis module test the data in order to detect the abnormal patterns or packets.

9. Alerts and statistics are displayed in real time through the interface.

10. Captured data can be stored for further study or analysis using external tools.

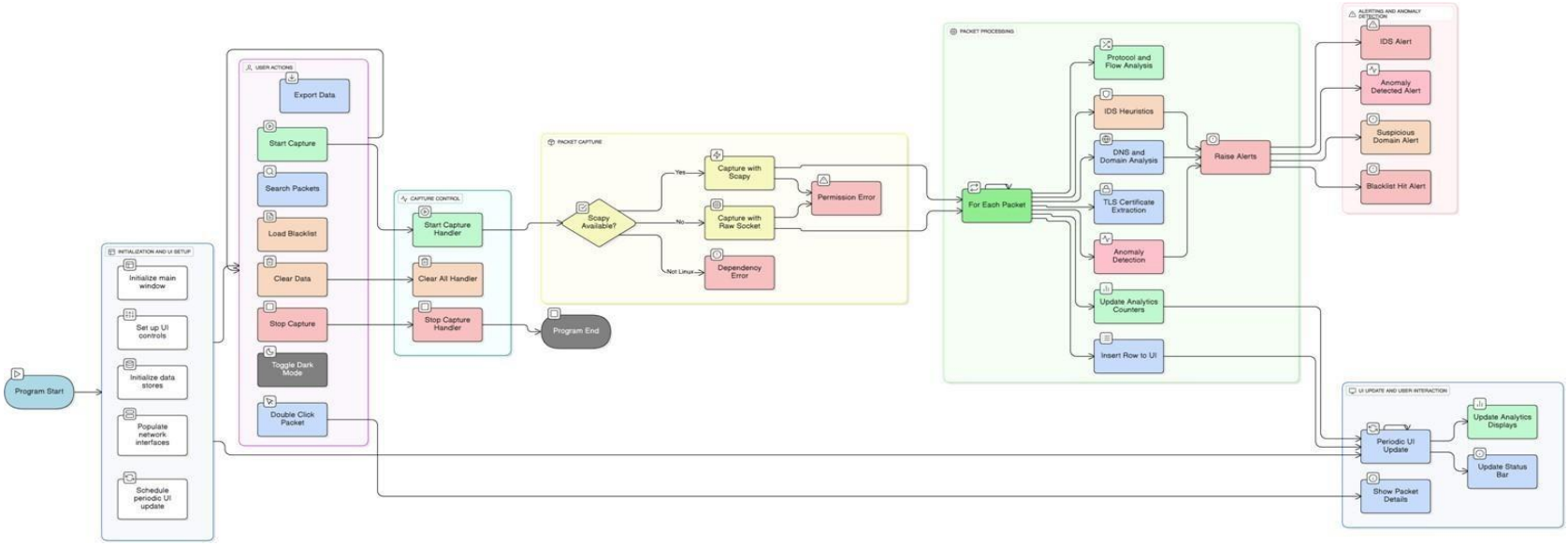


**Fig. 1.1** System architecture and workflow of the enhanced packet sniffer, including packet capture, protocol decoding, anomaly detection, alert generation, and interface updates.

## 2 Results Experimental Setup

The system was tested in both Linux and Windows operating system machines. Different attack scenarios were simulated which includes ARP spoofing, TCP SYN flooding, and generated DNS queries with random patterns.

**Observations**

- **ARP Spoofing Detection:** The system identified inconsistencies in MAC address mappings and generated alerts.
- **SYN Flood Detection:** A large number of incomplete TCP connections indicated potential flooding activity.
- **DNS Analysis:** Random and high-entropy domain names were successfully flagged as suspicious.
- **TLS Inspection:** Certificate details such as issuer and validity period were extracted for analysis.
- **Active Hosts:** The system highlighted the most active IP addresses based on traffic volume.

These results show that the tool is effective for demonstration purposes and controlled testing environments [6].

## Discussion

The Python-based packet sniffer is very useful and also easy to work. With the help of protocol decoding, anomaly detection, and a simple interface, the system maintains a balance between basic tools and more advanced, complex solutions. It gives clear information about network activity without confusing the user which is very helpful for beginners, students, and small-scale experiments. A major strength of the system is its modular design. This allows the system to be updated, improved, or expanded without any impact. This makes it more adaptable and easier to adapt or customize for various needs. However, it also has some drawbacks. While handling a large-scale network traffic the system face difficulty, this limits the use in real-world enterprise environments. Compared to other professional tools it also supports advanced protocols, which limits how detailed the analysis can be. Also, it is hard to analyze the encrypted traffic so there is a chance where some of the important network activities .

## Conclusion

This paper presents a Python-based advanced packet sniffer developed for both students and research purposes. It has multiple features such as packet capturing, protocol decoding, anomaly detection, DNS analysis, TLS inspection, and data visualization into a single lightweight tool. This mainly focuses on simplicity and accessibility, making it easy for beginners who are new to cybersecurity. At the same time, it is enough for researchers to experiment with different networks. By combining all these together, the tool reduces the complexity and helps users to analyze

It shows that the tool can successfully detect suspicious network activities in a system and also this says that even basic and simple tools are capable of providing a deep and clear understanding of cybersecurity concepts and related problems. It helps the users to observe real-time network behaviour and identify unusual patterns. Additionally, the system shows the capability of Python in developing practical and effective network monitoring solutions. Overall, the tool serves as a valuable learning resource and helps for the future improvements in cybersecurity research.

**Acknowledgments**

We would like to thank the Department of Computer Science and Engineering – Artificial Intelligence at Vignan's Institute of Information Technology for their guidance and support throughout this work.

## Disclosure of Interests

The authors declare that there are no conflicts of interest related to this research.

## References

1. Orebaugh, A., Ramirez, G., Beale, J.: *Wireshark & Ethereal Network Protocol Analyzer Toolkit*. Syngress (2006)
2. Shreya, K., Hayath Ali, S.: Network Traffic Analysis Using Sniffer. IJRASET (2025)

3. Al-Shaibany, N., Saif, A., Diyeb, I.: Ethical network surveillance using packet sniffing tools: A comparative study. IJCNIS (2018)
4. Rekhi, A., Saxena, A., Sharma, C., Pranav, A.: A Python-based Packet Sniffer Simulation for Network Traffic Analysis. Preprint (2023)
5. Scapy Project: http://www.scapy.net (Accessed September 2025)
6. Apri Siswanto., Abdul Syukur., Evizal Abdul Kadir., Suratin: Network Traffic Monitoring and Analysis Using Packet Sniffer IEEE(2019)
7. Suyash Garg: Creating an Effective Network Sniffer: IJTSRD(2020)